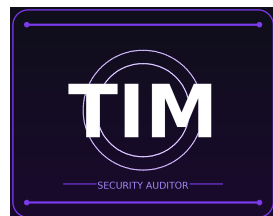

Tswap

Tim



13 May, 2026

Contents

Tswap	3
Table of Contents	3
Protocol Summary	4
Disclaimer	4
Risk Classification	4
Audit Details	5
Scope	5
Roles	5
Executive Summary	5
Issues found	5
Findings	5
High	5
[H-1] TswapPool::deposit is missing deadline check causing transaction too complete even after the deadline	5
[H-2] Incorrect fee in calculation in Tswapget::InputAmountBasedOnOutput causes protocol to take too many tokens from users resulting in lost fees . .	6
[H-3] Lack of slippage protection in TswapPool::swapExactOutput causes users to potentially receive way fewer tokens	7
[H-4] TswapPool::sellPoolTokens mismatches input and output tokens causing users to receive the incorrect amount of tokens	7
[H-5] In TswapPool::_swap the extra tokens given to users after every swapCount breaks the protocol invariant $fx * y = k$	8
Medium	10
Low	10
[L-1] TswapPool::LiquidityAdded event has parameters out of order	10
[L-2] default value returned by TswapPool::swapExactInput results in incorrect return value given	10
Gas	11

Informational	11
[I-1] PoolFactory::PoolFactory__PoolDoesNotExist is not used	11
[I-2] lacking zero address check	11
[I-3] PoolFactory::createPool should use .symbol() instead of name() . .	12

Tswap

Prepared by: Tim

Lead Auditor: Tim

Assisting Auditors: None

Date:

Version: 1.0

Table of Contents

- [Puppy Raffle audit](#)
- [Table of Contents](#)
- [Protocol Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Audit Details](#)
 - [Scope](#)
 - [Roles](#)
- [Executive Summary](#)
 - [Issues found](#)
- [Findings](#)
 - [High](#)
 - * [\[H-1\] TSwapPool::deposit is missing deadline check causing transactions to complete even after the deadline](#)
 - * [\[H-2\] Incorrect fee calculation in TSwapPool::getInputAmountBasedOnOutput causes protocol to take too many tokens from users, resulting in lost fees](#)
 - * [\[H-3\] Lack of slippage protection in TSwapPool::swapExactOutput causes users to potentially receive way fewer tokens](#)
 - * [\[H-4\] TSwapPool::sellPoolTokens mismatches input and output tokens causing users to receive the incorrect amount of tokens](#)
 - * [\[H-5\] In TSwapPool::_swap the extra tokens given to users after every swapCount breaks the protocol invariant of \$x * y = k\$](#)

- * [Low](#)
- * [\[L-1\] TSwapPool::LiquidityAdded event has parameters out of order](#)
- * [\[L-2\] Default value returned by TSwapPool::swapExactInput results in incorrect return value given](#)
- **Informationals**
 - * [\[I-1\] PoolFactory::PoolFactory__PoolDoesNotExist is not used and should be removed](#)
 - * [\[I-2\] Lacking zero address checks](#)
 - * [\[I-3\] PoolFacotry::createPool should use .symbol\(\) instead of .name\(\)](#)
 - * [\[I-4\] Event is missing indexed fields](#)

Protocol Summary

Protocol does X, Y, Z

Disclaimer

The team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the [CodeHawks](#) severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

Roles

Executive Summary

Issues found

Severity	Number of issues found
High	5
Medium	0
Low	2
gas	0
Informational	3
Total	10

Findings

High

[H-1] TswapPool::deposit is missing deadline check causing transaction too complete even after the deadline

Description: the `deposit` function accepts a deadline parameter, which according to the documentation is the “the deadline The deadline for the transaction to be completed by”. however this parameter is not used. as a consequence, operations that add liquidity to the pool might be executed at unexpected time , in market conditions where the deposit rate is unfavorable.

Impact: Transaction could be sent when market conditions are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept: The `deadline` parameter is unused.

Recommended Mitigation: consider making the following change to the function.

```
1 function deposit(
2     uint256 wethToDeposit,
3     uint256 minimumLiquidityTokensToMint,
4     uint256 maximumPoolTokensToDeposit,
5     uint64 deadline
6 )
7     external
8 +     revertIfDeadlinePassed(deadline)
9     revertIfZero(wethToDeposit)
10    returns (uint256 liquidityTokensToMint)
```

[H-2] Incorrect fee in calculation in Tswapget : : InputAmountBasedOnOutput causes protocol to take too many tokens from users resulting in lost fees

Description: The `getInputAmountBasedOnOutput` function is intended to calculate the amount of token a user should deposit given the amount of tokens of output tokens. However, the function currently miscalculates the resulting amount. when calculating the fee, it scales the amount by 10_000 instead of 1_000.

Impact: protocol takes more fees than expected

Proof of Concept:

Recommended Mitigation:

```
1     function getInputAmountBasedOnOutput(
2         uint256 outputAmount,
3         uint256 inputReserves,
4         uint256 outputReserves
5     )
6         public
7         pure
8         revertIfZero(outputAmount)
9         revertIfZero(outputReserves)
10        returns (uint256 inputAmount)
11    {
12        return
13 -        ((inputReserves * outputAmount) * 10_000) /
14 -        ((inputReserves * outputAmount) * 1_000) /
15        ((outputReserves - outputAmount) * 997);
16    }
```

[H-3] Lack of slippage protection in TswapPool::swapExactOutput causes users to potentially receive way fewer tokens

Description: The `swapExactOutput` function does not include any sort of slippage protection. this function is similar to what is done in `TswapPool::swapExactInput`, where the function specifies a `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: if the market conditions change before the transaction processes, the user could get a much worse swap.

Proof of Concept: 1. The price of 1 weth right now is 1000 USDC 2. user inputs a `swapExactOutput` looking for 1 weth 1. `inputToken` = USDC 2. `outputToken` = Weth 3. `outputAmount` = 1 4. `deadline` = whatever 3. The function does not offer a `maxInput` amount 4. As the transaction is pending in the mempool, the market changes. and the price moves huge -> 1 weth is now 10000 USDC. 10x more than the user expected 5. the transaction completes, but the user sent the protocol 10,000 USDC instead of the expected 1,000 USDC.

Recommended Mitigation: we should include a `maxInputAmount` so the user only has to spend up to a specific amount, and can predict how much they will spend on the protocol.

```
1 function swapExactOutput{
2     IERC20 inputToken
3 +     uint256 maxInputAmount
4 .
5 .
6 .
7     inputAmount = getInputAmountBasedOnOutput(
8         outputAmount,
9         inputReserves,
10        outputReserves
11    );
12 +     if(inputAmount > maxInputAmount) {
13 +         revert();
14 +     }
15
16     _swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-4] TswapPool::sellPoolTokens mismatches input and output tokens causing users to receive the incorrect amount of tokens

Description: the `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive weth in exchange. users indicate how many pool tokens they are willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculates the swapped amount. This is due to the fact that the `swapExactOutput` function is called, whereas the `swapExactInput`

function is the one that should be called. Because users specify the exact amount of input tokens, not output.

Impact: users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality

Proof of Concept:

Recommended Mitigation: consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note this would also require changing the `sellTokens` function to accept a new parameter (ie `minWethToReceive` to be passed to `swapExactInput`)

```

1  function sellPoolTokens(
2      uint256 poolTokenAmount
3  +      minWethToReceive
4  ) external returns (uint256 wethAmount) {
5  -     return swapExactOutput(
6  -         i_poolToken,
7  -         i_wethToken,
8  -         poolTokenAmount,
9  -         uint64(block.timestamp)
10     );
11  +     return
12  +         swapExactOutput(
13  +             i_poolToken,
14  +             poolTokenAmount,
15  +             i_wethToken,
16  +             minWethToReceive
17  +             uint64(block.timestamp)
18     );

```

[H-5] In TswapPool :: _swap the extra tokens given to users after every swapCount breaks the protocol invariant $x * y = k$.

Description: The protocol follows a strict invariant of $x * y = k$. where: - x : The balance of the pool token - y The balance of Weth - k The constant product of the two balance

This means, that whenever the balance changes in the protocol, the ratio between the two amounts should remain constant, hence the k . However, this is broken due to the extra incentive in the `_swap` function. Meaning that over time the protocol funds will be drained. The following block of code is responsible for the issue.

```

1  swap_count++;
2  if (swap_count >= SWAP_COUNT_MAX) {
3      swap_count = 0;
4      outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
5  }

```

Impact: A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol. Most simply put, the protocol's core invariant is broken.

Proof of Concept: 1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens

2. That user continues to swap until all the protocol funds are drained

Proof of code

place the following into `Tswap.t.sol`

```
1 function testInvariantBroken() public {
2     vm.startPrank(liquidityProvider);
3     weth.approve(address(pool), 100e18);
4     poolToken.approve(address(pool), 100e18);
5     pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
6     vm.stopPrank();
7
8     uint256 outputWeth = 1e17;
9
10    vm.startPrank(user);
11    poolToken.approve(address(pool), type(uint256).max);
12    poolToken.mint(user, 100e18);
13    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
14    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
15    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
16    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
17    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
18    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
19    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
20    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
21    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
22
23    int256 startingY = int256(weth.balanceOf(address(pool)));
24    int256 expectedDeltaY = int256(-1) * int256(outputWeth);
25
26    pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
27    vm.stopPrank();
28
29    uint256 endingY = weth.balanceOf(address(pool));
```

```

30         int256 actualDeltaY = int256(endingY) - int256(startingY);
31         assertEq(actualDeltaY, expectedDeltaY);
32     }

```

Recommended Mitigation: Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the $x * y = k$ protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```

1 -         swap_count++;
2 -         // Fee-on-transfer
3 -         if (swap_count >= SWAP_COUNT_MAX) {
4 -             swap_count = 0;
5 -             outputToken.safeTransfer(msg.sender, 1
6 -             _000_000_000_000_000_000);

```

Medium

Low

[L-1] TswapPool::LiquidityAdded event has parameters out of order

Description: when the `LiquidityAdded` event is emitted in the `Tswap::_addLiquidityMintedAndTransfer` function, it logs values in an incorrect order. The `poolTokensDeposit` value should go in the third parameter position, whereas the `wethToDeposit` value should go second.

Impact: Event emission is incorrect, leading to offchain functions potentially malfunctioning

```

1 -         emit LiquidityAdded(msg.sender, poolTokensToDeposit,
2 -         wethToDeposit);
2 +         emit LiquidityAdded(msg.sender, wethToDeposit,
3 +         poolTokensToDeposit);

```

[L-2] default value returned by TswapPool::swapExactInput results in incorrect return value given

Description: The `SwapExactInput` function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value `input` it is never assigned a value, nor uses an explicit return statement

Impact: the return value will always be 0, giving incorrect information to the caller.

Proof of Concept:**Recommended Mitigation:**

```
1  {uint256 inputReserves = inputToken.balanceOf(address(this));
2      uint256 outputReserves = outputToken.balanceOf(address(this));
3
4  -      uint256 outputAmount = getOutputAmountBasedOnInput(
5  +      uint256 output = getOutputAmountBasedOnInput(
6      inputAmount,
7      inputReserves,
8      outputReserves
9      );
10
11 -      if (outputAmount < minOutputAmount) {
12 +      if (output < minOutputAmount) {
13 -          revert TSwapPool__OutputTooLow(outputAmount,
14 +          revert TSwapPool__OutputTooLow(output, minOutputAmount);
15      }
16
17 -      _swap(inputToken, inputAmount, outputToken, outputAmount);
18 +      _swap(inputToken, inputAmount, outputToken, output);
19  }
```

Gas**Informational****[I-1] PoolFactory::PoolFactory__PoolDoesNotExist is not used**

```
1  - error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] lacking zero address check

```
1  constructor(address wethToken) {
2  +  if(wethToken == address(0)) {
3  +      revert();
4  +  }
5  i_wethToken = wethToken;
6  }
```

[I-3] PoolFactory::createPool should use .symbol() instead of name()

```
1 -         string memory liquidityTokenSymbol = string.concat("ts",  
    IERC20(tokenAddress).name());  
2 +         string memory liquidityTokenSymbol = string.concat("ts",  
    IERC20(tokenAddress).symbol());
```